

M2 — Broker Integration & Sync

Module Specification

Trading Journal App · v1.0 · July 2026 · Status: ready to break into tickets

Field	Value
Module ID	M2
Track / owner	Data In track — one owner, end to end
Depends on	M0 Foundation & Shared Contracts (data model, design system, error contract) · M1 Identity (authenticated user, timezone hint)
Blocks	M3 Trade Data Core cannot normalise, score, or session-tag a single trade until M2 hands it raw facts.
Scope in	MT5 and cTrader connections, credential storage and encryption, multi-account and prop-firm support, historical backfill, live incremental sync, connection health and re-auth, CSV/statement fallback import, manual single-trade entry, deduplication
Scope out	Trade normalisation, metrics and outlier detection (M3) · strategy tagging (M4) · dashboard visualisation (M6) · notification delivery, only triggering (M9) · account-count entitlement limits, only enforcement (M11)

Purpose

M2 answers one question: how does a trader's real trading activity get into the app without them typing it in? It is the “waiter” between a broker platform and the journal — it fetches trades from MT5 or cTrader, or accepts them from a CSV, and hands clean raw trade facts to M3. M2 never interprets a trade; it only proves the trade happened and gets out of the way.

Components in this specification

1. Stories
2. Data flow and schema
3. UI screens
4. Detailed description and business logic
5. Flowcharts
6. Test plan
7. Quality benchmarks
8. Security constraints
9. Error handling

10. External dependencies
11. Performance and scalability
12. Module relationships
13. Data policies (EU and US)
14. Documentation

1. Stories

Fourteen stories cover the module. Each becomes an epic or a ticket; the acceptance criteria are the testable conditions, not restatements of the story.

ID	Story	Acceptance criteria	Pri
US-01	As a trader, I want to connect my MT5 account so my trades import automatically.	Investor-password connection tests successfully, status becomes connected, and a backfill job is queued within 5 s.	Must
US-02	As a trader, I want to connect my cTrader account via OAuth so I don't have to type in my trades.	Authorization Code + PKCE flow completes, account list is fetched, and the chosen account(s) move to connected.	Must
US-03	As a trader with a prop-firm account, I want to tag it as such so it's treated correctly everywhere else.	account_type = prop_firm is stored and visible on every screen that lists the connection.	Must
US-04	As a trader with multiple accounts, I want to connect more than one so all my trading shows in one place.	Any number of BrokerConnection rows per user; each syncs independently; each is individually manageable.	Must
US-05	As a new user, I want my full trade history imported once so I don't lose past performance data.	Backfill fetches all history the broker exposes, writes RawTrade rows, and emits broker.backfill_completed.	Must
US-06	As an active trader, I want new trades to appear automatically without me refreshing.	Scheduled incremental sync runs per connection; new trades appear within the target latency in Section 7.	Must
US-07	As a trader whose broker session breaks, I want to be told clearly and asked to reconnect.	Failed auth sets status = needs_reauth, M9 is notified, and a one-tap reconnect flow is offered.	Must
US-08	As a trader on a platform without an API, I want to upload a CSV so I can still use the journal.	CSV/XLSX upload with column mapping; common MT5/cTrader export formats auto-detected.	Must
US-09	As a trader, I want duplicate trades to never appear	Dedup key blocks re-insertion; a duplicate row is	Must

	twice, even across re-syncs or re-uploads.	a silent no-op, never a visible error.	
US-10	As a trader, I want to see sync status and last-synced time for each account.	Every connection shows status, last_synced_at and next scheduled sync, updated live.	Should
US-11	As a trader, I want to disconnect an account without losing my historical journal.	Disconnect revokes credentials and stops sync immediately; all past RawTrade rows are kept, unchanged.	Must
US-12	As a trader, I want my broker credentials stored securely.	Tokens are encrypted at rest, never appear in API responses, and never appear in logs.	Must
US-13	As a trader, I want to manually trigger a re-sync if I think something's missing.	"Sync now" is rate-limited (Section 7), shows progress, and cannot be used to hammer the broker API.	Should
US-14	As a trader, I want to log a single trade from a platform I haven't connected.	Manual entry writes directly into the same RawTrade contract M3 consumes, tagged source = manual.	Should

Ticket-splitting note

US-01, US-05 and US-06 form the smallest shippable slice for one broker — connect, backfill once, then keep syncing. Build and demo that end to end before adding the second broker, CSV, or manual entry.

2. Data flow and schema

Six entities. M2 is the sole owner of every one of them; no other module writes to these tables, and other modules reference a connection only by connection_id or a trade only by its RawTrade id.

Entity	Purpose	Key fields
BrokerConnection	One connected account. The thing a trader sees and manages.	id · user_id · broker (mt5 ctrader) · account_type (live demo prop_firm) · broker_account_id · display_label · status · connected_at · last_synced_at · next_sync_at · disconnected_at
BrokerCredential	The secret half of a connection, stored	id · connection_id ·

	apart from it for tighter access control.	encrypted_access_token · encrypted_refresh_token · token_expires_at · scope
SyncJob	One run of fetching trades — backfill or incremental.	id · connection_id · type (backfill incremental manual) · status · started_at · finished_at · trades_fetched · trades_new · error_code
RawTrade	M2's entire output contract. A clean, unopinionated trade fact.	id · connection_id (nullable for manual/CSV) · broker_trade_id · symbol · side · volume · open_time · close_time · open_price · close_price · fees · swap · source · dedup_hash · raw_payload · imported_at
ImportBatch	One CSV/statement upload and its outcome.	id · user_id · connection_id (nullable) · filename · column_mapping · status · rows_total · rows_imported · rows_skipped_duplicate · rows_failed
ConnectionEvent	Audit log of everything that happens to a connection.	id · connection_id · event_type (connected reauth_required error disconnected sync_failed) · detail · created_at

2.1 How data moves

Connecting. The screen posts credentials (MT5) or completes an OAuth round trip (cTrader). M2 creates a BrokerConnection (status = connecting), a BrokerCredential, and a ConnectionEvent, then queues a SyncJob of type backfill. Nothing downstream reacts until the first RawTrade rows exist.

Backfill. The job fetches all available history, converts each broker record into the RawTrade shape, computes a dedup_hash for each, and inserts new rows only. On completion it emits broker.backfill_completed and flips status to connected.

Incremental sync. A scheduler creates a SyncJob of type incremental per active connection on a fixed cadence (Section 7). It fetches records since last_synced_at, de-duplicates, inserts what's new, and emits broker.trade_imported for the batch — the signal M3 subscribes to.

CSV import. An ImportBatch is created on upload, parsed against the confirmed column mapping, and each row becomes a RawTrade with source = csv after the same dedup check as a live sync. There is no ongoing SyncJob for an ImportBatch — it is one-shot by nature.

Manual entry. A single RawTrade is written directly with source = manual and connection_id = null. It skips SyncJob and ImportBatch entirely — there is nothing to schedule or dedup against a broker for a trade the broker never saw.

The one rule that keeps this clean

M2 never computes P&L, sessions, strategy tags, or outliers — it only produces clean raw trade facts, each

carrying a source so it's always traceable. M3 is the only module allowed to interpret them.

3. UI screens

Ten screens, all built as working responsive mockups in M2_UI_Mockups.html — open it in any browser. Each screen can be switched between its states and between mobile and desktop widths, so the build target is unambiguous.

ID	Screen	States built
M2-S01	Broker connections hub	Empty · Has connections · Loading
M2-S02	Choose a connection method	Default
M2-S03	Connect MT5	Default · Testing · Error
M2-S04	Connect cTrader	Redirecting · Pick account · Denied / error
M2-S05	Label the account	Default · Saving
M2-S06	Historical backfill progress	In progress · Complete · Partial error
M2-S07	Connection detail & sync status	Connected · Syncing · Needs re-auth · Error
M2-S08	CSV / statement import	Upload · Map columns · File error
M2-S09	Manual trade entry	Default · Saving · Error
M2-S10	Disconnect a broker account	Confirm · Disconnected

3.1 Rules that apply to every screen

- Every screen implements all its states. A screen without a designed empty, loading and error state is not done.
- All copy is sentence case, active voice, and never blames the trader. Errors say what happened and what to do next.
- Nothing is lost on failure: form values in S03, S05 and S09 survive an error, always.
- Keyboard-navigable end to end, with visible focus. Contrast at least 4.5:1.
- Responsive from 375 px to desktop. Both are first-class, neither is an afterthought.
- Colour is reserved: green/amber/red only ever describe connection status, never used decoratively.

4. Detailed description and business logic

The rules below are the module's real content. Where a rule looks unusually strict, the reasoning is stated — those are the ones that get quietly dropped otherwise.

4.1 Connecting and authenticating

Rule	Decision
MT5 access path	Investor (read-only) password plus login ID and server. This

	is a research spike, not a settled integration — MT5 exposes no first-party public API for third parties; access typically requires a bridge or an investor-password terminal connection. Treat the timeline as unknown until the spike concludes, and build CSV import first so the app works regardless of the outcome.
cTrader access path	OAuth 2.0 Authorization Code flow with PKCE against the cTrader Open API. This is a settled, documented integration and carries materially less risk than MT5.
Credentials are read-only by design	Neither integration is ever granted trade-placement scope. A connection can only read history — it can never place, modify, or close a trade. This is stated to the trader on-screen (S02), not just in a policy.
Multiple accounts per user	Unlimited BrokerConnection rows per user in M2 itself; any ceiling on count is an M11 entitlement check applied at creation time, not a limit M2 enforces on its own.
Prop-firm accounts	Not a different connection mechanism — a label (account_type = prop_firm) applied on the same MT5/cTrader connection. This is the deliberate fix for the competitor gap noted in the product's founding research: prop accounts must be first-class, not bolted on.

4.2 Sync scheduling

Rule	Decision
Backfill	Runs once per connection, immediately on creation, as an asynchronous job. Never blocks the UI (Section 11).
Incremental cadence	Every 5 minutes per active connection by default, adjustable per broker if rate limits require it. A webhook or streaming source, where a broker offers one, supplements but never replaces the scheduled tick.
Backoff on failure	Up to 3 retries per tick with exponential backoff before a SyncJob is marked failed. Repeated failure (3 consecutive ticks) moves the connection to status = error and notifies the trader once, not on every tick.
Token refresh	Attempted silently before every sync using the stored refresh token. Only a refresh failure moves the connection to needs_reauth — an access-token expiry alone is not user-visible.
Manual “sync now”	Rate-limited to 1 per connection per 2 minutes, to protect both the broker API and the trader's own rate limits.

4.3 Deduplication

Rule	Decision
Dedup key	connection_id + broker_trade_id where the broker provides a stable trade identifier. Where it doesn't (some CSV exports), a computed hash of symbol + side + volume + open_time + close_time + open_price stands in.
Re-sync safety	Fetching a time range that overlaps a previous sync is always safe — the dedup key makes re-insertion a no-op rather than a duplicate row.
CSV re-upload	Checked against every existing RawTrade for the user, not just prior CSV imports — a trader who later connects MT5 live must not see their earlier CSV history duplicated.
Edited-then-resynced trades	If a broker allows a trade record to change after close (rare, but happens with commission adjustments), the incoming record with the same dedup key updates the existing row rather than inserting a second one.

4.4 Connection lifecycle

State	Meaning and transitions
disconnected	Starting state, and the state after a trader disconnects. No credentials stored. Historical RawTrade rows from this connection, if any, are untouched.
connecting	Credentials submitted or OAuth in progress. Transient — resolves to connected or back to disconnected within seconds.
connected	Normal state. Scheduled sync runs on cadence.
syncing	A sync is actively in flight. Purely informational — functionally identical to connected once it finishes.
needs_reauth	Token refresh failed. Sync is paused, not retried blindly, until the trader reconnects.
error	Repeated fetch failures with a valid token — typically a broker-side outage. Sync retries continue in the background; the trader is notified once.

5. Flowcharts

Three diagrams: how a broker account gets connected, what happens on every scheduled sync, and the states a connection can occupy. Each is followed by the reasoning behind its branches.

5.1 Connecting a broker account

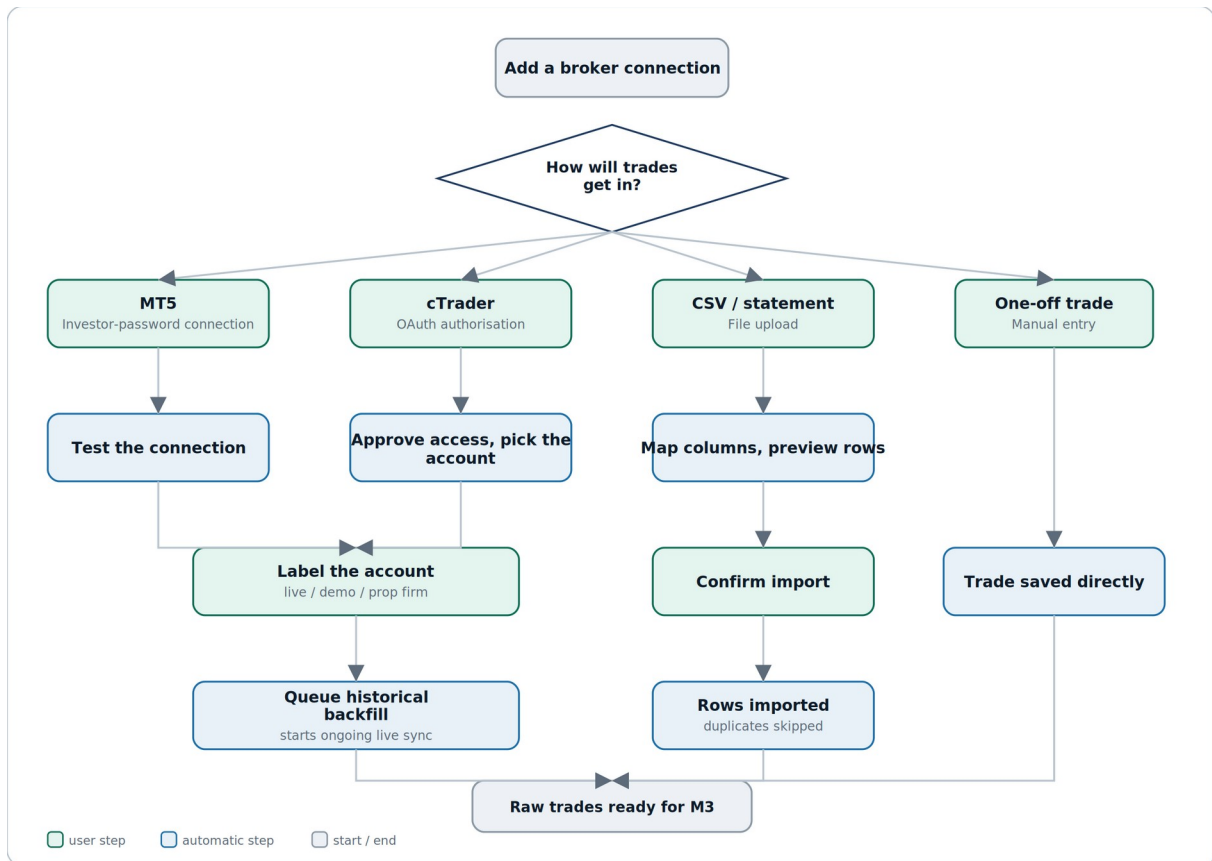


Figure 1 — from “add a connection” to trades ready for M3.

Reading it. The one real decision is which of four paths a trader takes. MT5 and cTrader both end up as live, ongoing connections and share the “label the account” step because both need it equally; CSV and manual entry are one-shot and never need a live-sync label in the same sense, so they route around it.

Why MT5 and cTrader converge but CSV doesn't. Labelling live/demo/prop-firm matters most for a connection that will keep producing trades for months — it drives filters and rules everywhere downstream. A CSV import is a single event; asking for the same classification adds a step without a matching ongoing payoff, so it's folded into a lighter “confirm import” instead.

5.2 The sync cycle

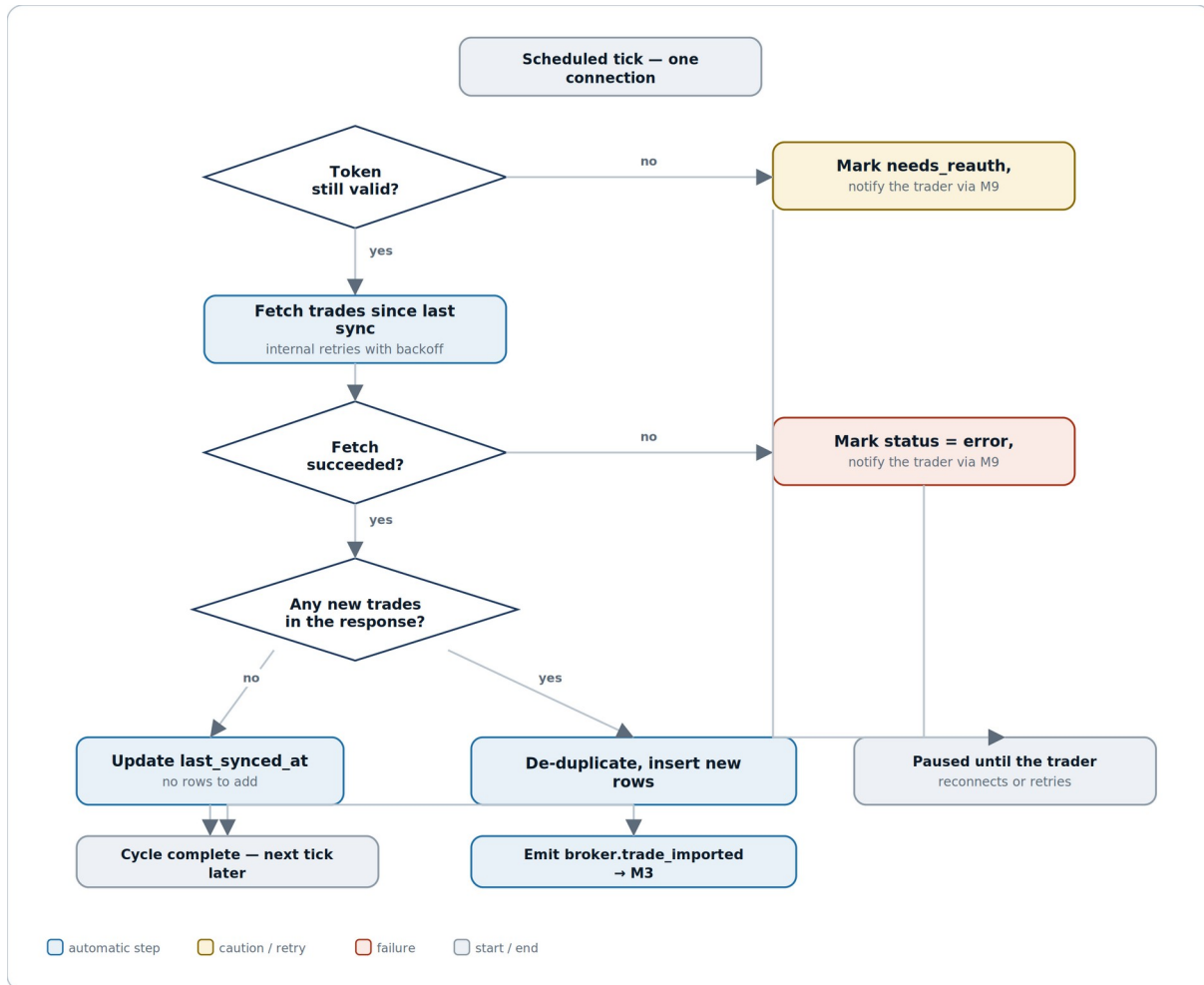


Figure 2 — what happens on every scheduled tick, including failure paths.

Reading it. Two gates come before any data is touched: is the token valid, and did the fetch succeed. Both failures route to the same outcome — the trader is notified once and the cycle pauses for that connection — but they're kept as separate gates because they have different fixes: `needs_reauth` is fixed by the trader reconnecting, error usually resolves on its own once the broker recovers.

Why “no new trades” is its own branch, not an error. A quiet trading day is the most common outcome of any given tick, and it isn't a failure of anything — the diagram treats it as a first-class, unremarkable path that simply updates the watermark and moves on.

5.3 Connection lifecycle

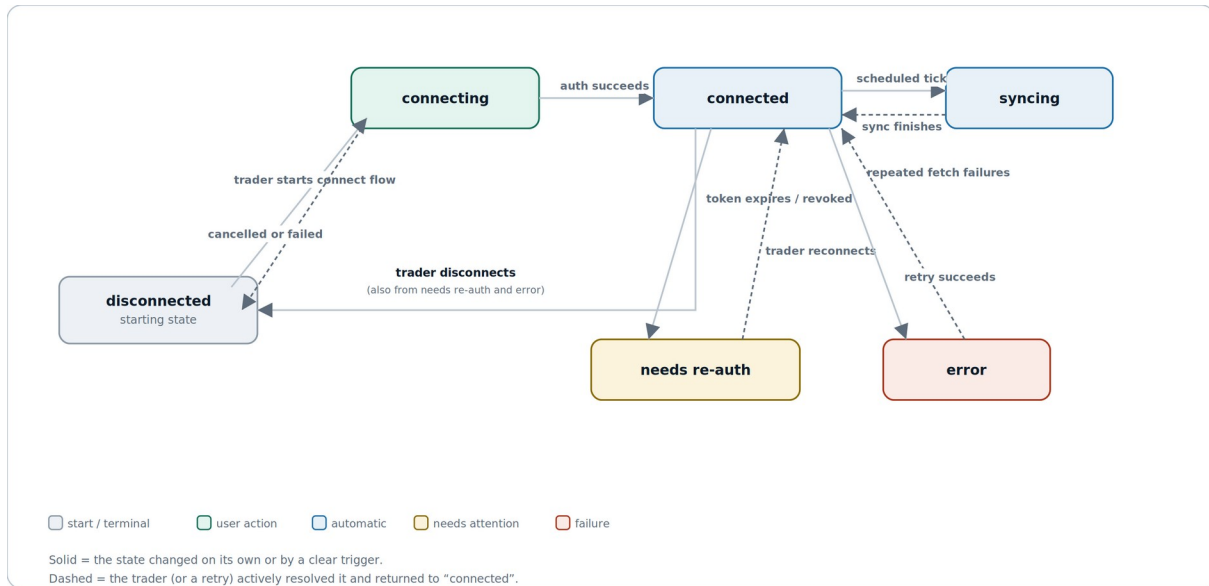


Figure 3 — the states a connection can be in, and what moves it between them.

Reading it. Solid arrows are things that happen to a connection — a token expiring, repeated fetch failures, a scheduled tick starting. Dashed arrows are things a trader (or a retry) does about it — reconnecting, or a retry quietly succeeding. Every non-terminal state can reach disconnected the same way, because disconnecting is always the trader's escape hatch regardless of what state a connection is stuck in.

Why needs_reauth and error are kept separate. They look similar — both pause syncing — but conflating them would hide who's responsible for the fix. needs_reauth is waiting on the trader; error is waiting on the broker (or a retry). Merging them would either nag traders about outages they can't fix, or stay silent about a token only they can renew.

6. Test plan

A sync pipeline fails in ways that are invisible until a trader's numbers are quietly wrong, so correctness here is tested at every layer, not just the happy path.

Layer	What it covers	Key cases
Unit	Pure logic with no I/O.	Dedup hash computation · session-agnostic field mapping · backoff timing · column-mapping validation
Integration	The module against its own contract and store.	Full connect → backfill → RawTrade write · incremental sync writes only new rows · CSV import respects mapping and dedup
Contract	What M2 promises other modules.	broker.trade_imported / broker.backfill_completed fire with the agreed shape · RawTrade fields match M3's expected input exactly
End-to-end	Real trader journeys through the UI.	Connect MT5 → label → backfill → trades visible · cTrader OAuth round trip → account picker → connected · CSV upload → map → import → no duplicates on re-upload
Resilience	Broker-side and network failure paths.	Token expiry mid-sync → needs_reauth · broker outage → error after 3 failed ticks → recovers automatically once the broker returns · partial backfill → reported honestly, not silently dropped
Security	Credential handling and abuse paths.	Tokens never appear in logs or API responses · OAuth state parameter rejected if tampered · CSV upload rejects non-tabular or oversized files

6.1 Manual QA checklist

#	Manual QA checklist item
1	Every screen renders correctly at 375 px and at 1440 px.
2	Every state in the mockup (loading, error, empty, success) is reachable in the real build.
3	Disconnecting mid-sync stops the in-flight job cleanly, not just future ticks.

4	Reconnecting after needs_reauth resumes sync without re-importing existing trades.
5	A CSV re-uploaded twice produces zero duplicate RawTrade rows.
6	Backfill on a large seeded account (5,000+ trades) never blocks the UI thread.
7	Manual trade entry appears in the journal identically to a synced trade, correctly labelled as manual.
8	No credential, token, or investor password appears in any log or error message.
9	Sync-now is refused with a clear message when called twice within the rate-limit window.
10	A prop-firm tag set at connect time is still correct after a reconnect.

6.2 Test data

- Seeded connections in every status: connecting, connected, syncing, needs_reauth, error, disconnected.
- One account with 5,000+ trades to exercise backfill performance and pagination.
- A CSV fixture with an intentionally malformed row, to confirm partial-failure handling doesn't abort the whole import.
- Two connections under one cTrader OAuth login, to exercise the account picker.
- A prop-firm connection and a demo connection, to confirm labelling survives reconnects and re-syncs.

7. Quality benchmarks

Numbers, not adjectives. Anything unmeasurable is not a benchmark.

Metric	Target	How it is measured
Incremental sync latency	New broker trade visible in RawTrade within 6 minutes, p95	Time from broker execution timestamp to RawTrade.imported_at
Backfill throughput	10,000 trades imported within 10 minutes	Job duration vs trade count, sampled per completed SyncJob
Connect flow completion	p95 under 8 s from credential submit to connected (excluding OAuth provider time)	Server timing, connect endpoint only
CSV import (5,000 rows)	p95 under 60 s from confirm to complete	Job completion time vs confirm time
Sync reliability	≥ 99% of scheduled ticks complete without manual intervention	SyncJob success rate over trailing 7 days
Dedup accuracy	0 duplicate RawTrade rows across any re-sync or re-upload in QA	Automated dedup regression suite
False needs_reauth rate	< 1% of re-auth prompts resolved by the trader re-entering unchanged credentials	Correlate reconnect events with credential diff
Availability	99.5% monthly for connect and sync endpoints	Uptime monitor

Why sync reliability is the number that matters most

A trader who stops trusting the sync stops opening the app — the entire product proposition (“your trades capture themselves”) collapses the moment a trader has to double-check whether something came through. This is the metric to protect above almost anything else in M2.

8. Security constraints

M2 holds the keys to a trader's live brokerage account. Everything below is a requirement, not a recommendation.

Area	Constraint
Credential storage	Access and refresh tokens, and MT5 investor passwords, are encrypted at rest with a per-record key from a managed KMS. Plaintext credentials are never stored, logged, or emailed.

Credential transmission	HTTPS everywhere with HSTS. Credentials are never carried in a URL query string or in a log line, structured or not.
Credential separation	BrokerCredential is a separate table from BrokerConnection with tighter row-level access — a query for connection status never needs to touch the secret.
OAuth	Authorization Code flow with PKCE. The state parameter is validated on callback. Redirect URIs are allow-listed exactly, no wildcards.
Read-only scope	Every OAuth grant and MT5 credential requests the narrowest scope that allows reading history. Trade-placement scope is never requested.
Rate limiting	Per user and per connection on connect, sync-now, and CSV upload, to protect both the broker API and M2's own infrastructure.
Deletion on disconnect	Credentials are deleted, not deactivated, at the moment of disconnect — a stored token for an account the trader no longer wants connected is pure liability.
Logging	Never log tokens, passwords, or full raw broker payloads containing account numbers. Structured logs redact known credential fields by pattern, not by convention alone.
Audit trail	Every connect, disconnect, re-auth, and status change is recorded as a ConnectionEvent.
File upload safety	CSV/XLSX uploads are size-capped, type-validated, and parsed in a sandboxed job — never executed or evaluated as anything but tabular data.

9. Error handling

Every error has a code for logs, a defined system behaviour, and copy the trader actually sees. Copy is written here, not improvised during the build.

Code	Scenario	System behaviour	What the trader sees
BRK-001	Wrong MT5 login, password, or server	Connection attempt rejected, nothing stored	"We couldn't connect. Check the server name and investor password, then try again."
BRK-002	MT5 investor password used is actually a trading password	Rejected where detectable; otherwise connects but is flagged for review	"That looks like a trading password. Use the investor (read-only) password instead."
BRK-003	cTrader OAuth denied or cancelled by the trader	No connection created	"Connection wasn't completed. Try again, or import a CSV instead."
BRK-004	cTrader OAuth state mismatch (possible CSRF)	Request rejected outright, logged as a security event	"Something went wrong with that connection attempt. Please try again."
SYN-001	Token refresh fails	Status → needs_reauth, sync paused, trader notified once via M9	"Access expired. Reconnect to keep your trades syncing."
SYN-002	3 consecutive sync ticks fail with a valid token	Status → error, retries continue with backoff, trader notified once	"Sync has been failing. We'll keep trying — you can also try now."
SYN-003	Broker rate limit hit mid-sync	Backs off beyond the broker's stated window, resumes automatically	No trader-facing message unless it persists past SYN-002's threshold
IMP-001	CSV file unreadable or wrong format	Rejected before parsing	"We couldn't read that file. Make sure it's a .csv or .xlsx export and try again."
IMP-002	Required column unmapped at confirm	Blocked, mapping screen re-shown	"Symbol, side, open/close time and volume all need to be mapped before importing."
IMP-003	Row fails validation during import (e.g. close before open)	Row skipped, import continues, count reported	"{n} rows couldn't be imported — see details." with a downloadable list
DUP-001	Incoming trade matches an existing dedup hash	Silent no-op, not surfaced as an error	Nothing — by design (see Section 4.3)

MAN-001	Manual trade has close time before open time	Rejected client- and server-side	"Close time can't be before open time."
SYS-001	Broker API fully unavailable	SyncJob retries with backoff; connection eventually marked error if sustained	"Sync has been failing." (same as SYN-002)
SYS-002	Backfill job crashes partway	Partial results kept, job marked partial, resumable from last successful page	"Imported with a few gaps — {n} of {total} trades came through."

The rule behind the honest-gap messages

BRK-002 and SYS-002 are deliberately specific rather than vague, unlike the anti-enumeration pattern used in M1's auth errors. There is no attacker to protect against here — only a trader trying to trust their own numbers — so precision helps rather than hurts.

10. External dependencies

Every dependency needs a defined failure mode. A third party being down must degrade the module, never break it.

Dependency	Used for	If it fails	Fallback
cTrader Open API	OAuth connection and trade history for cTrader accounts	New cTrader connections and syncs fail	CSV import stays fully available; existing connections retry automatically on backoff
MT5 access path (spike)	Reading trade history from MT5 accounts	New MT5 connections and syncs fail; this is the single largest unknown in the module	CSV import is the guaranteed path and must not depend on this spike succeeding
Job scheduler / queue	Backfill jobs, incremental sync ticks, CSV parsing	No new trades sync; existing data is untouched	Alert on-call; queued jobs resume once the scheduler recovers
KMS / secrets encryption	Encrypting stored credentials	New connections can't be created; existing ones can't refresh tokens	Fail closed — never fall back to storing a credential unencrypted
M9 Notifications	Delivering needs_reauth, error, and backfill-complete alerts	Trader isn't told a connection needs attention	M2 owns the intent and retries the notification request; M2's own status badges (S01/S07) remain accurate regardless
M1 Identity	Authenticated user context for every connection	No connection can be created or managed	Not a real failure mode in practice — M1 is a hard dependency, not a degrade-gracefully one
M11 Billing	Entitlement check for number of connections allowed	Unknown whether a new connection should be blocked	Fail open for connecting (never block a trader's data over a billing hiccup); reconcile and flag the account afterwards
Object storage	Holding uploaded CSV/XLSX files during processing	CSV import can't be parsed	Upload stays queued, trader is told it's still processing rather than shown a hard failure

Fail open or fail closed

Security checks fail closed — if credentials can't be encrypted, the connection is refused. Convenience and billing checks fail open — if M11's entitlement check is unreachable, a trader's data still gets connected and synced, and the discrepancy is reconciled after the fact. Confusing the two either locks out every trader over an outage, or

quietly disables a security control.

11. Performance and scalability

Concern	Position
Scale assumption for v1	Up to 20,000 active connections and 200,000 incremental sync ticks per day. Nothing in M2 should need re-architecting below that.
Hottest path	The scheduled incremental sync tick, running per active connection on a fixed cadence. It must scale horizontally by connection, not by user.
Deliberate slow path	Backfill on a large account is expected to take minutes, not seconds — it runs as a background job with visible progress (S06), never inline with the connect request.
Broker API is the real bottleneck	M2's own infrastructure is rarely the limiting factor; per-broker rate limits are. Sync scheduling must be broker-aware and back off before hitting a broker's own limits, not just react after a 429.
CSV parsing is asynchronous	Uploads are queued and parsed by a background job with progress, identical in shape to a backfill job — large files must never block a web request.
What breaks first	The scheduler/queue under a spike of newly-connected accounts all backfilling at once (e.g. a marketing push). Backfill jobs should be throttled per-tenant so one trader's large account can't starve everyone else's sync tick.
Growth triggers	Revisit design at 100,000 active connections, sustained incremental sync latency above the Section 7 target, or any single broker's rate limit becoming a binding constraint across many users at once.

12. Module relationships

This is the contract surface. Once agreed, other modules can build against it with mocks while M2 is still being written.

Direction	Module	What crosses the boundary
Exposes	M3 Trade Core	Clean RawTrade rows plus the broker.trade_imported and broker.backfill_completed events. M3 must never reach into a broker API itself.
Exposes	M6 Dashboard	Connection status and per-account trade counts, so account filters can be built without M6 knowing anything about brokers.
Exposes	M11 Billing	Connection count per user, for entitlement checks against plan limits.
Consumes	M1 Identity	Authenticated user_id for every connection, and the platform hint captured during M1's first-run setup, used to pre-suggest a connection method.
Consumes	M0 Foundation	Shared data model, design system, and error contract.
Consumes	M9 Notifications	Delivery of needs_reauth, sync-error, and backfill-complete alerts. M2 decides what to send; M9 sends it.
Consumes	M11 Billing	Entitlement answer for “can this user add another connection.” Fails open per Section 10.
Emits	—	broker.connected · broker.backfill_completed · broker.trade_imported · broker.needs_reauth · broker.error · broker.disconnected
Triggers	M3, M4, M6	On broker.trade_imported, M3 normalises new rows; M4 can re-evaluate rule adherence; M6 refreshes any open dashboard.

The dependency to watch

M3 cannot normalise a trade it never receives, and every downstream module — dashboards, strategy tagging,

the AI coach — is a consumer of M3, not of M2 directly. Agree the exact RawTrade field names, types, and nullability in M0 before either team builds against a guess. This is the single most consequential handoff in the whole Data In track.

13. Data policies (EU and US)

M2 holds brokerage credentials and a trader's full trading history — among the most sensitive data the product touches. Building to these rules now is far cheaper than retrofitting them.

13.1 What we hold and why

Data	Classification	Lawful basis (GDPR)
Broker credentials (tokens, investor password)	Sensitive — grants access to a third-party financial account	Contract — needed to provide the sync the trader explicitly requested
Trade history (symbol, size, price, timing, P&L)	Personal data; commercially sensitive financial data	Contract
Account labels (prop firm name, display label)	Personal data, low sensitivity	Contract
CSV/XLSX files during processing	May contain the same trade history plus whatever else the file includes	Contract, for the duration of processing only
Connection and sync event logs	Personal data	Legitimate interests — security, reliability, and abuse prevention

13.2 GDPR (EU / UK)

Obligation	How M2 meets it
Lawful basis (Art. 6)	Contract, documented per data type above. No connection is ever created without the trader's own action.
Data minimisation (Art. 5(1)(c))	Only read-only scopes are ever requested. Trading-capable credentials, if a trader mistakenly provides them, are never knowingly used to place or modify trades.
Security of processing (Art. 32)	Encryption at rest for all credentials, encryption in transit everywhere, and the access separation described in Section 8.
Storage limitation	Credentials are deleted immediately on disconnect, not retained “just in case.” Uploaded CSV files are deleted after processing completes (Retention schedule, 13.4).
Right of access / portability (Art. 15/20)	Trade history held by M2 is included in M1's account-wide export — M2 does not maintain a separate export path.
Erasure (Art. 17)	On account deletion (via M1), M2 must revoke every credential immediately and erase or anonymise RawTrade rows on the same 30-day grace period M1 defines. M2 honours <code>user.deletion_requested</code> as a contract, not an option.

Processor agreements (Art. 28)	Required with the broker-integration platform if a third-party aggregator is ever used, and with cloud/KMS providers. No processor is used without one.
International transfers	Standard Contractual Clauses or an adequacy decision required for any processor outside the EEA, particularly relevant since broker APIs may route through non-EEA infrastructure.
Breach notification (Art. 33/34)	A leaked broker credential is a high-severity incident — it exposes a third-party financial account, not just app data. The runbook (Section 14) must treat this as an immediate, not routine, escalation.

13.3 United States (CCPA / CPRA and state law)

Requirement	How M2 meets it
Sensitive personal information	Financial account credentials and trading history are sensitive PI under CPRA. Use is limited to providing the sync the trader requested — no secondary use, no sale.
Right to know / access	Same account-wide export as GDPR access, via M1.
Right to delete	Same deletion flow as GDPR erasure, via M1; M2 revokes credentials immediately regardless of the grace period on the rest of the account.
Right to correct	A trader can edit an account's label and type (S05/S07) at any time; trade data itself is corrected by re-sync or manual entry, not by editing broker-sourced facts directly.
Notice at collection	S02 and S03/S04 state plainly, at the point of connecting, what will be read and that it is read-only — not buried in a policy.
Not investment advice	M2 only imports what a trader already did. It never recommends, scores, or advises on a trade at the point of import — that framing is preserved for M7 to handle carefully, not introduced here.
State breach-notification laws	Same high-severity treatment as GDPR Art. 33/34 — a compromised broker credential triggers the incident runbook regardless of which jurisdiction the trader is in.

13.4 Retention schedule

Data	Retention	Reason
Active connection credentials	Life of the connection	Needed to sync; nothing is retained

		beyond that need
Credentials after disconnect	Deleted immediately, not retained	A credential for a connection the trader ended is pure liability
RawTrade history	Life of account + 30-day grace (matches M1)	A trader's trading history is core product value, not a log to be pruned early
Uploaded CSV/XLSX files	Deleted within 24 hours of successful processing	The file itself is a transient input; only the resulting RawTrade rows need to persist
SyncJob and ConnectionEvent logs	12 months	Investigation and reliability analysis
Failed/incomplete import batches	30 days, then purged	Long enough to support troubleshooting, short enough not to accumulate stale sensitive files

Two things not to get wrong

First: disconnect must delete credentials immediately, every time, with no code path that leaves a stale token behind — this is tested explicitly in Section 6. Second: on account deletion, M2's erasure must genuinely propagate; if M1 erases a user but M2 still holds their RawTrade rows or a live credential, the product has not honoured the deletion it promised.

14. Documentation

Documentation is part of the module, not an afterthought once the code works.

Deliverable	Contents
Module README	What M2 does, how to run it locally, environment variables for broker API credentials, how to seed test connections.
Contract / API reference	Every endpoint and event M2 exposes, generated from the contract agreed in Section 12 and M0.
Decision log (ADRs)	Initial entries: why MT5 is a spike and CSV is the guaranteed fallback; why credentials live in a separate table from connections; why prop-firm is a label, not a separate flow; why dedup falls back to a computed hash without a broker trade ID.
Runbook	What to do when a broker API is down, when sync latency spikes, when a credential leak is suspected (tie to the 72-hour GDPR clock), and how to manually resume a stuck backfill.
Data inventory & retention register	The tables in Section 13, maintained as living documents rather than a one-off.
User-facing help	Articles: generating an MT5 investor password · what “read-only” actually means · why a connection needs re-auth · preparing a CSV for import · disconnecting safely.
UI mockups	M2_UI_Mockups.html — all 10 screens with every state, responsive at mobile and desktop widths.

14.1 Definition of done

M2 is done when all of the following are true:

- Every story meets its acceptance criteria.
- The contract in Section 12 is published, and at least one other module has built against it — even if only with mocks.
- All ten screens are built with every state, responsive at 375 px and desktop.
- Every error in Section 9 is reachable and shows its written copy.
- Security constraints in Section 8 are implemented and tested, including the credential-deletion-on-disconnect case.
- Disconnect and (via M1) deletion both work end to end, and erasure genuinely propagates.
- The MT5 access-path spike has concluded with a documented outcome, even if the outcome is “CSV-only for now.”
- Benchmarks in Section 7 are measured, not assumed.
- README, ADRs and runbook are written; the data inventory is filled in.
- Demoed to the other two owners, with the RawTrade contract walked through explicitly.

End of M2 specification — v1.0. Same 14-component structure as M1; built to be the second reference point when the pattern repeats across M3–M12.